

```

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


dataset = pd.read_csv(r'C:\Users\user\FSDS_Adv_Gene_AI_Agentic_AI\Machine
Learning\Regression\Polynomial\emp_sal.csv')


X = dataset.iloc[:,1:2]
Y = dataset.iloc[:, 2]


from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()

lin_reg.fit(X,Y)


plt.scatter(X, Y, color = 'red')

plt.plot(X, lin_reg.predict(X), color = 'blue')

plt.title('Linear regression Graph')

plt.xlabel('Position level')

plt.ylabel('Salary')

plt.show()


lin_model_pred = lin_reg.predict([[6.5]])

lin_model_pred


# Polynomial Model

from sklearn.preprocessing import PolynomialFeatures

poly_reg = PolynomialFeatures()

X_poly = poly_reg.fit_transform(X)


poly_reg = PolynomialFeatures()

```

```
X_poly = poly_reg.fit_transform(X)
```

```
poly_reg.fit(X_poly,Y)
```

```
lin_reg_2 = LinearRegression()
```

```
lin_reg_2.fit(X_poly, Y)
```

```
print(lin_reg) # Linear model with 1 degree poly
```

```
print(poly_reg) # Poly model with 2 degree
```

```
print(lin_reg_2) # Linear model with 2 degree
```

```
plt.scatter(X, Y, color = 'red')
```

```
plt.plot(X, lin_reg_2.predict(poly_reg.fit_transform(X)), color = 'blue')
```

```
plt.title('Truth or Bluff (Polynomial Feature)')
```

```
plt.xlabel('Position level')
```

```
plt.ylabel('Salary')
```

```
plt.show()
```

```
poly_model_pred = lin_reg_2.predict(poly_reg.fit_transform([[6.5]]))
```

```
poly_model_pred
```

```
poly_reg = PolynomialFeatures(degree = 3)
```

```
X_poly = poly_reg.fit_transform(X)
```

```
poly_reg.fit(X_poly,Y)
```

```
lin_reg_3 = LinearRegression()
```

```
lin_reg_3.fit(X_poly, Y)
```

```
print(lin_reg) # Linear model with 1 degree poly
```

```
print(poly_reg) # Poly model with 3 degree
```

```
print(lin_reg_2) # Linear model with 2 degree
```

```
plt.scatter(X, Y, color = 'red')
plt.plot(X, lin_reg_3.predict(poly_reg.fit_transform(X)), color = 'blue')
plt.title('Truth or Bluff (Polynomial Feature)')
plt.xlabel('Position level')
plt.ylabel('Salary')
plt.show()
```

```
poly_model_pred = lin_reg_3.predict(poly_reg.fit_transform([[6.5]]))
poly_model_pred
```

```
poly_reg = PolynomialFeatures(degree = 4)
X_poly = poly_reg.fit_transform(X)
poly_reg.fit(X_poly,Y)
```

```
lin_reg_4 = LinearRegression()
lin_reg_4.fit(X_poly, Y)
```

```
print(lin_reg) # Linear model with 1 degree poly
print(poly_reg) # Poly model with 4 degree
print(lin_reg_2) # Linear model with 2 degree
```

```
plt.scatter(X, Y, color = 'red')
plt.plot(X, lin_reg_4.predict(poly_reg.fit_transform(X)), color = 'blue')
plt.title('Truth or Bluff (Polynomial Feature)')
plt.xlabel('Position level')
plt.ylabel('Salary')
plt.show()
```

```
poly_model_pred = lin_reg_4.predict(poly_reg.fit_transform([[6.5]]))
poly_model_pred
```

```
poly_reg = PolynomialFeatures(degree = 5)
```

```
X_poly = poly_reg.fit_transform(X)
```

```
poly_reg.fit(X_poly,Y)
```

```
lin_reg_5 = LinearRegression()
```

```
lin_reg_5.fit(X_poly, Y)
```

```
print(lin_reg) # Linear model with 1 degree poly
```

```
print(poly_reg) # Poly model with 5 degree
```

```
print(lin_reg_2) # Linear model with 2 degree
```

```
plt.scatter(X, Y, color = 'red')
```

```
plt.plot(X, lin_reg_5.predict(poly_reg.fit_transform(X)), color = 'blue')
```

```
plt.title('Truth or Bluff (Polynomial Feature)')
```

```
plt.xlabel('Position level')
```

```
plt.ylabel('Salary')
```

```
plt.show()
```

```
poly_model_pred = lin_reg_5.predict(poly_reg.fit_transform([[6.5]]))
```

```
poly_model_pred
```

```
poly_reg = PolynomialFeatures(degree = 6)
```

```
X_poly = poly_reg.fit_transform(X)
```

```
poly_reg.fit(X_poly,Y)
```

```
lin_reg_6 = LinearRegression()
```

```
lin_reg_6.fit(X_poly, Y)
```

```
print(lin_reg) # Linear model with 1 degree poly
```

```
print(poly_reg) # Poly model with 5 degree
```

```
print(lin_reg_2) # Linear model with 2 degree
```

```
plt.scatter(X, Y, color = 'red')  
plt.plot(X, lin_reg_6.predict(poly_reg.fit_transform(X)), color = 'blue')  
plt.title('Truth or Bluff (Polynomial Feature)')  
plt.xlabel('Position level')  
plt.ylabel('Salary')  
plt.show()
```

```
poly_model_pred = lin_reg_6.predict(poly_reg.fit_transform([[6.5]]))  
poly_model_pred
```

```
# SVR model prediction  
from sklearn.svm import SVR  
svr_regressor = SVR()  
svr_regressor.fit(X,Y)
```

```
svr_model_pred = svr_regressor.predict([[6.5]])  
svr_model_pred
```

```
svr_regressor = SVR(kernel = 'rbf', degree = 3, gamma='scale')  
svr_regressor.fit(X,Y)  
svr_model_pred = svr_regressor.predict([[6.5]])  
svr_model_pred
```

```
svr_regressor = SVR(kernel = 'sigmoid', degree = 3, gamma='scale')  
svr_regressor.fit(X,Y)  
svr_model_pred = svr_regressor.predict([[6.5]])  
svr_model_pred
```

```
svr_regressor = SVR(kernel = 'poly', degree = 4, gamma='scale')  
svr_regressor.fit(X,Y)
```

```
svr_model_pred = svr_regressor.predict([[6.5]])
```

```
svr_model_pred
```

```
svr_regressor = SVR(kernel = 'poly', degree = 4, gamma='auto')
```

```
svr_regressor.fit(X,Y)
```

```
svr_model_pred = svr_regressor.predict([[6.5]])
```

```
svr_model_pred
```

```
svr_regressor = SVR(kernel = 'poly', degree = 5, gamma='scale')
```

```
svr_regressor.fit(X,Y)
```

```
svr_model_pred = svr_regressor.predict([[6.5]])
```

```
svr_model_pred
```

```
# Knn model
```

```
from sklearn.neighbors import KNeighborsRegressor
```

```
knn_regressor = KNeighborsRegressor()
```

```
knn_regressor.fit(X, Y)
```

```
knn_model_pred = knn_regressor.predict([[6.5]])
```

```
knn_model_pred
```

```
knn_regressor = KNeighborsRegressor(n_neighbors= 5, weights='uniform', p=2)
```

```
knn_regressor.fit(X, Y)
```

```
knn_model_pred = knn_regressor.predict([[6.5]])
```

```
knn_model_pred
```

```
knn_regressor = KNeighborsRegressor(n_neighbors= 5, weights='distance', p=1)
```

```
knn_regressor.fit(X, Y)
```

```
knn_model_pred = knn_regressor.predict([[6.5]])
```

```
knn_model_pred
```

```
knn_regressor = KNeighborsRegressor(n_neighbors= 4, weights='uniform', p=2)
```

```
knn_regressor.fit(X, Y)
knn_model_pred = knn_regressor.predict([[6.5]])
knn_model_pred
```

```
knn_regressor = KNeighborsRegressor(n_neighbors= 4, weights='distance', p=1)
knn_regressor.fit(X, Y)
knn_model_pred = knn_regressor.predict([[6.5]])
knn_model_pred
```

```
knn_regressor = KNeighborsRegressor(n_neighbors= 3, weights='distance', p=1)
knn_regressor.fit(X, Y)
knn_model_pred = knn_regressor.predict([[6.5]])
knn_model_pred
```

```
knn_regressor = KNeighborsRegressor(n_neighbors= 3, weights='uniform', p=1)
knn_regressor.fit(X, Y)
knn_model_pred = knn_regressor.predict([[6.5]])
knn_model_pred
```

```
knn_regressor = KNeighborsRegressor(n_neighbors= 2, weights='distance', algorithm= 'auto', p=2)
knn_regressor.fit(X, Y)
knn_model_pred = knn_regressor.predict([[6.5]])
knn_model_pred
```

```
knn_regressor = KNeighborsRegressor(n_neighbors= 2, weights='uniform', algorithm= 'auto', p=2)
knn_regressor.fit(X, Y)
knn_model_pred = knn_regressor.predict([[6.5]])
knn_model_pred
```

```
# Decision Tree Algorithm
from sklearn.tree import DecisionTreeRegressor
```

```
dt_regressor = DecisionTreeRegressor()
dt_regressor.fit(X, Y)
dt_model_pred = dt_regressor.predict([[6.5]])
dt_model_pred
```

```
# Random Forest
```

```
from sklearn.ensemble import RandomForestRegressor
rf_regressor = RandomForestRegressor(random_state=0, n_estimators = 8)
rf_regressor.fit(X,Y)
rf_model_pred = rf_regressor.predict([[6.5]])
rf_model_pred
```